

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#01)

Apply modern simulation tools ModelSim and Logisim to implement basic processor components

Student Name: M. Ashnab Safdar Roll No:24I-6500 Section: CE-A

Date performed: 18/8/2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task, basic logic gates such as AND, OR, NOT, NAND, NOR, XOR, and XNOR is implemented in Verilog using structural (gate-level) modeling and In this task I have to write Verilog code and test it using testbench for the basic gates as we have done during the lab demonstration of the lab 01.

Code:

```
module LogicGates(a,b,out_and, out_or,out_not,out_nor,out_nand,out_xor,out_xnor);
```

```
input a;  
input b;
```

```
output out_and, out_or,out_not,out_nor,out_nand,out_xor,out_xnor;
```

```
and a1(out_and,a,b);  
or o1(out_or,a,b);  
not n1 (out_not,a);  
nor n2 (out_nor,a,b);  
nand n3(out_nand,a,b);  
xor x1(out_xor,a,b);  
xnor x2(out_xnor,a,b);
```

```
endmodule
```

```
module testbench(); //neverpass any argument to testbench
```

```
reg x;  
reg y;  
wire z_and,z_or,z_not,z_nor,z_nand,z_xnor,z_xor;
```

```
LogicGates uut(x,y,z_and,z_or,z_not,z_nor,z_nand,z_xor,z_xnor);
```

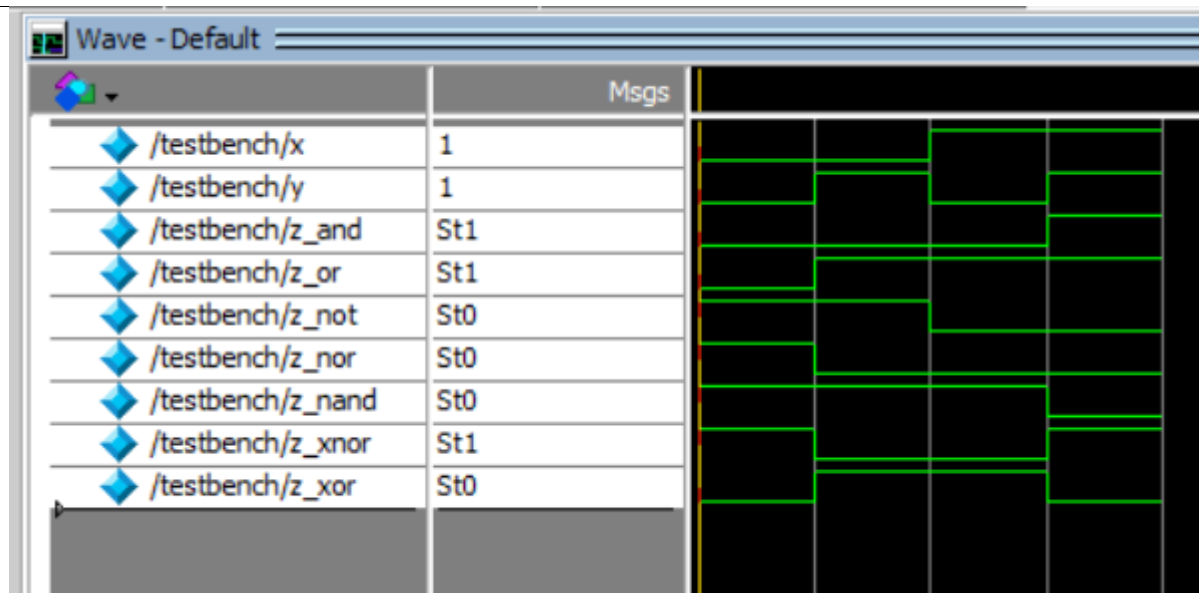
```
initial  
begin
```

```
    x=0;y=0;  
    #50 x=0;y=1;  
    #50 x=1;y=0;  
    #50 x=1;y=1;
```

```
end
```

endmodule

Output:



Task 2:

Description:

In this task, basic logic gates such as AND, OR, NOT, NAND, NOR, XOR, and XNOR is implemented in Verilog using data flow modeling and In this task I have to write Verilog code and test it using testbench for the basic gates as we have done during the lab demonstration of the lab 01.

Code:

```
module LogicGates(a,b,out_and, out_or,out_not,out_nor,out_nand,out_xor,out_xnor);

input a;
input b;

output out_and, out_or,out_not,out_nor,out_nand,out_xor,out_xnor;

assign out_and = a&b;
assign out_or = a|b;
assign out_not = ~a;
assign out_nor = ~(a|b);
assign out_nand = ~(a&b);
assign out_xor=a^b;
assign out_xnor=~(a^b);

endmodule
```

```
module testbench(); //neverpass any argument to testbench

reg x;
reg y;
wire z_and,z_or,z_not,z_nor,z_nand,z_xnor,z_xor;

LogicGates uut(x,y,z_and,z_or,z_not,z_nor,z_nand,z_xor,z_xnor);

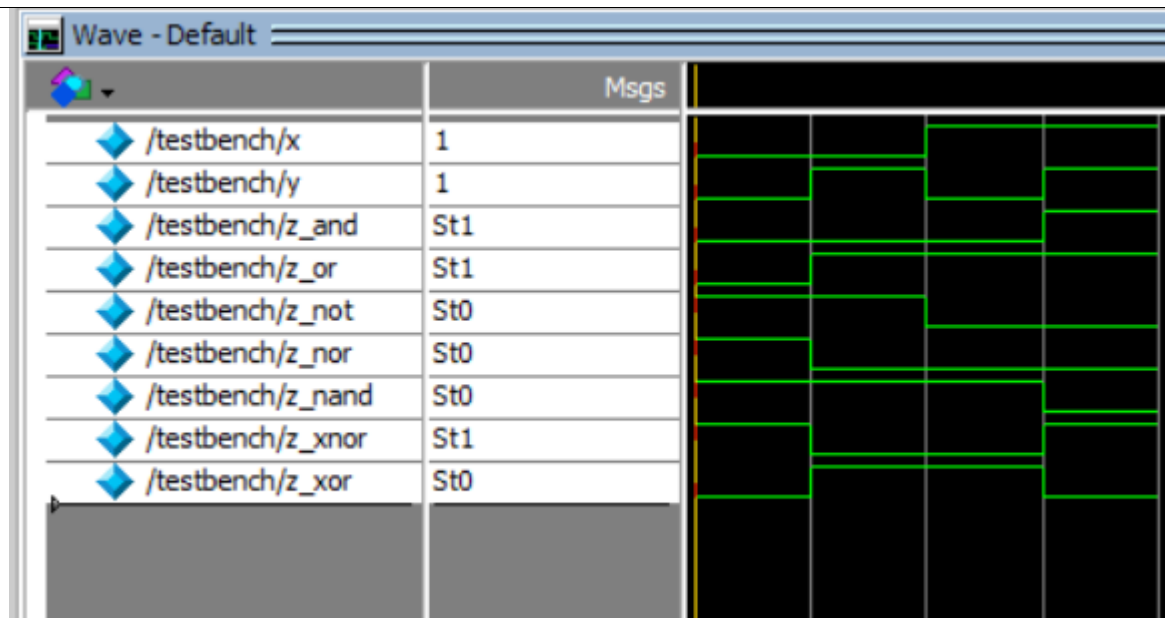
initial
begin

    x=0;y=0;
    #50 x=0;y=1;
    #50 x=1;y=0;
    #50 x=1;y=1;

end

endmodule
```

Output:



Task 3:

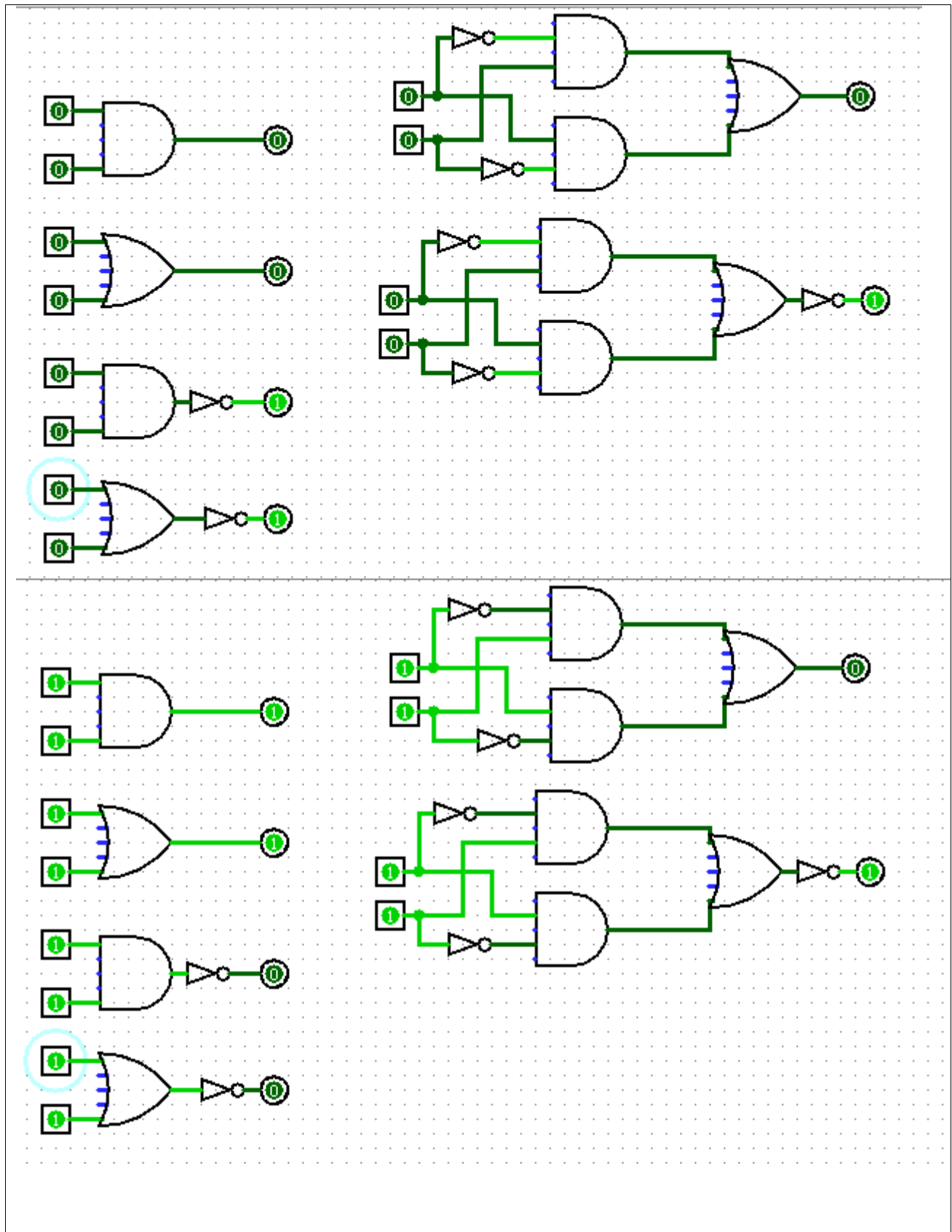
Description:

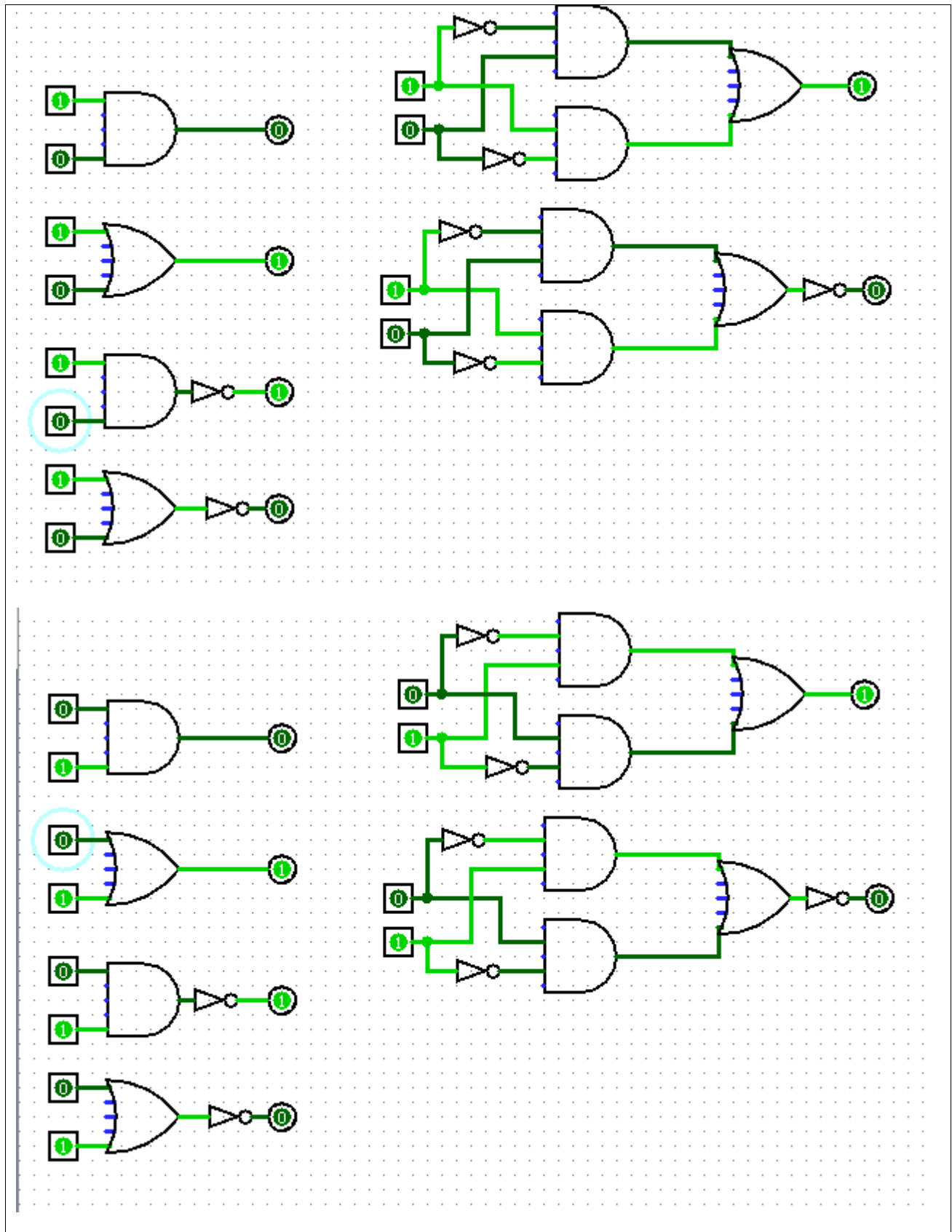
In this task the basic logic gates AND, OR, NOT, NAND, NOR, XOR, XNOR that was coded in Verilog in above two questions also built visually using Logisim which is basically a graphical digital logic design and simulation tool.

Code:

N/A

Output:





Task 4:

Description:

In this task I have to make a xor gate implementation as per given in lab manual picture of that I had to write an the code in modelsim and also had to check it using the testbench which is also pasted in the code below.

Code:

```
module xor_circuit(a,b,out);

input a,b;
output out;
wire an,bn,z1,z2;

not n1(an,a);
not n2(bn,b);

and a1(z1,an,b);
and a2(z2,bn,a);

or o(out,z1,z2);

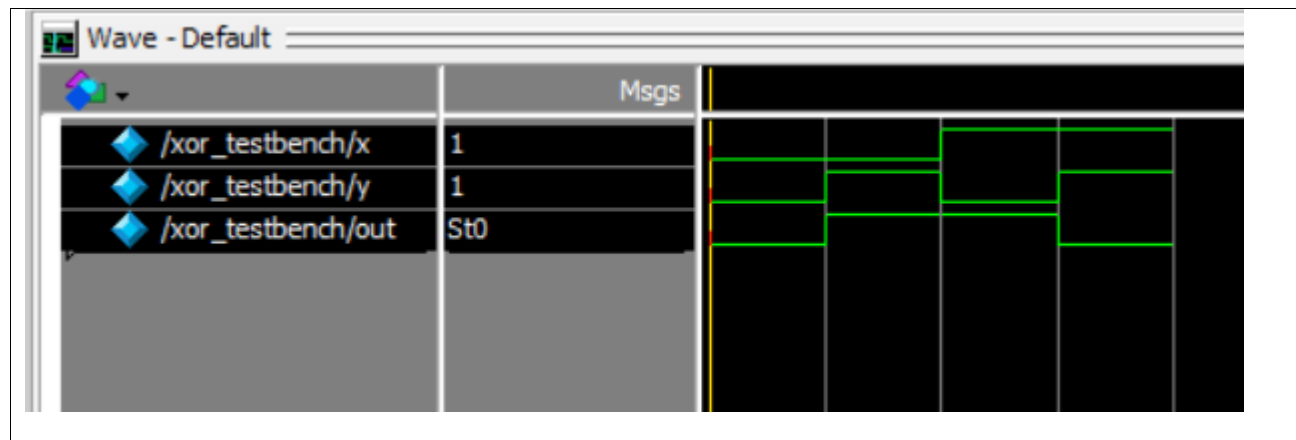
endmodule

module xor_testbench();

reg x;
reg y;
wire out;
xor_circuit uut(x,y,out);

initial
begin
    x=0;y=0;
    #50 x=0;y=1;
    #50 x=1;y=0;
    #50 x=1;y=1;
end
endmodule
```

Output:



Task 5:

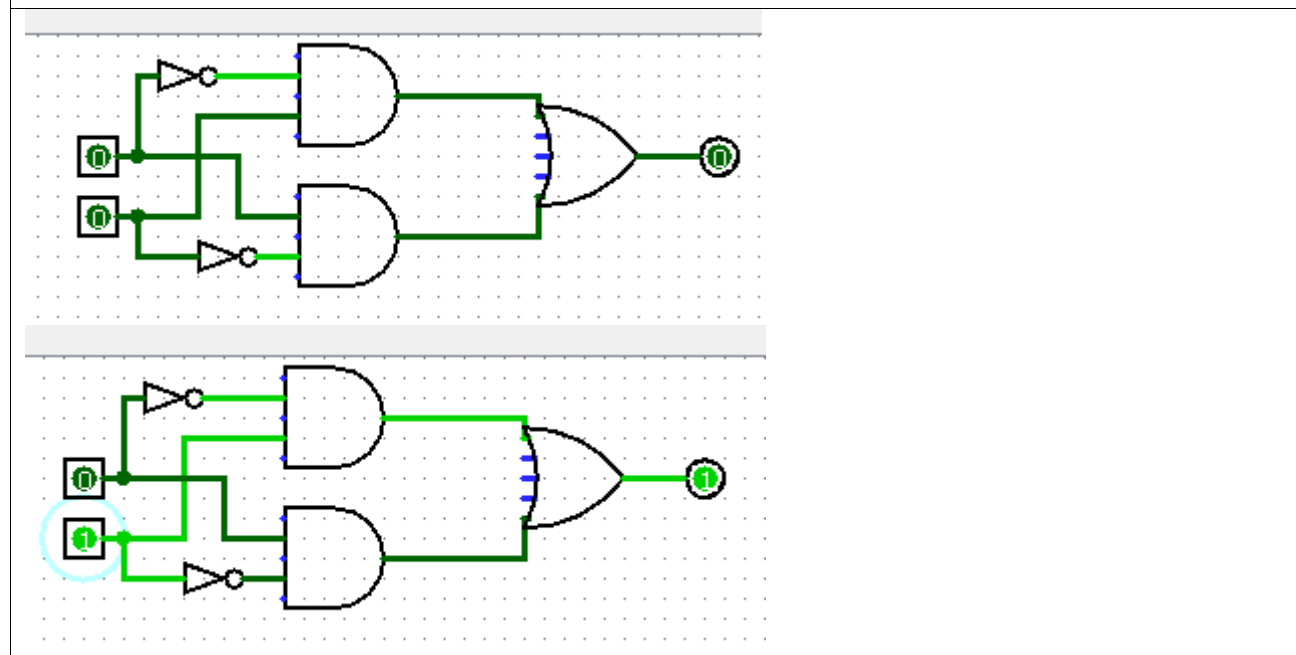
Description:

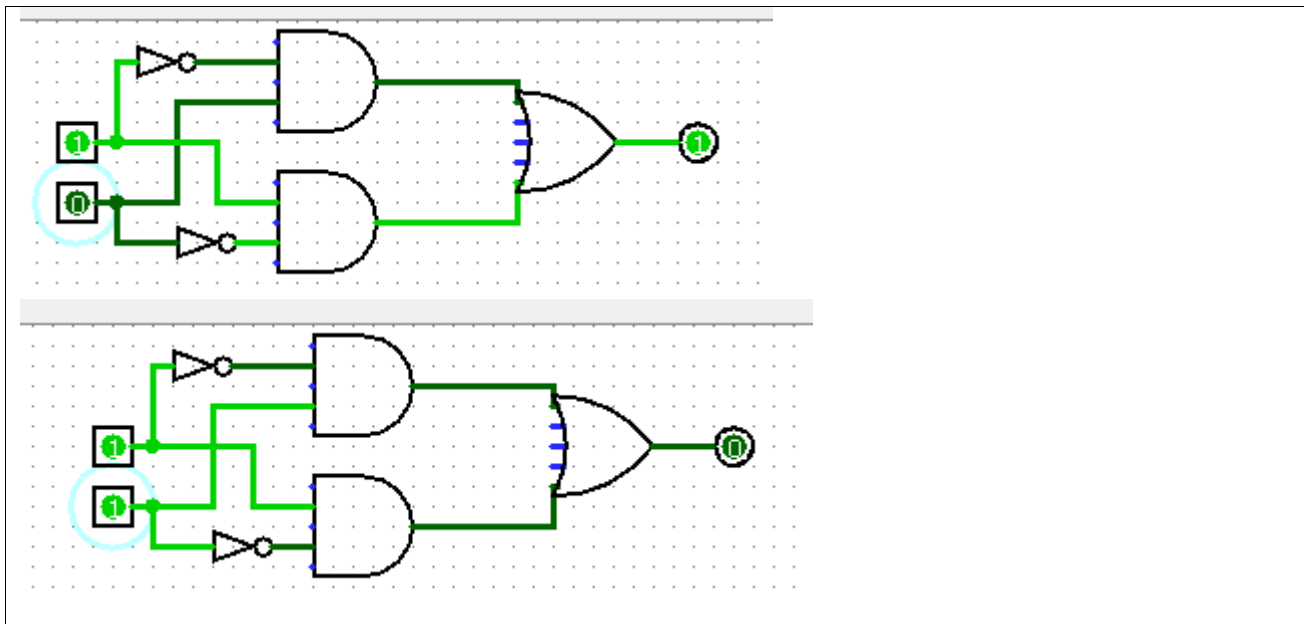
In this task I have to make a xor gate implementation as per given in lab manual picture of that I had to make the same circuit in Logisim and also had to check it using the different (all possible) inputs using the inputs A and B as per mentioned in the lab manual

Code:

N/A

Output:





Task 6:

Description:

In this task I had to make a code that give us one's complement of the 4 bit input provided also had to be done using both the gate level and dataflow level and then check it using the test bench module which is also provided in the code below.

Code:

```
module complement_1(a,b,c,d,na,nb,nc,nd);
input a,b,c,d;
output na,nb,nc,nd;

//not n1(na,a);
//not n2(nb,b);
//not n3(nc,c);
//not n4(nd,d);

assign na=~a;
assign nb=~b;
assign nc=~c;
assign nd=~d;

endmodule

module complement_1_testbench ();

reg w,x,y,z;
wire a,b,c,d;
```

```
complement_1 uut(w,x,y,z,a,b,c,d);
initial
begin
w=0; x=0; y=0; z=0;
#50 w=0; x=0; y=0; z=1;
#50 w=0; x=0; y=1; z=0;
#50 w=0; x=0; y=1; z=1;
#50 w=0; x=1; y=0; z=0;
#50 w=0; x=1; y=0; z=1;
#50 w=0; x=1; y=1; z=0;
#50 w=0; x=1; y=1; z=1;
#50 w=1; x=0; y=0; z=0;
#50 w=1; x=0; y=0; z=1;
#50 w=1; x=0; y=1; z=0;
#50 w=1; x=0; y=1; z=1;
#50 w=1; x=1; y=0; z=0;
#50 w=1; x=1; y=0; z=1;
#50 w=1; x=1; y=1; z=0;
#50 w=1; x=1; y=1; z=1;
#50;

end
endmodule
```

Output:

